LLM4Crypto: A Large Language Model-Based Cryptocurrency Market Sentiment Analysis System

Jiawei Guo aweiaaaaa/llm4crypto_test

Abstract

Cryptocurrency markets are highly volatile and information-sensitive, making them strongly influenced by social-media sentiment, market events, and the behavior of key opinion leaders (KOLs). This paper presents LLM4Crypto, a modular sentiment analysis system that integrates multi-source data collection, data cleaning, feature engineering, and large language model (LLM)-based structured analysis into a single end-to-end pipeline. The current implementation supports targeted collection from Twitter/X KOLs, CoinGecko market data, CoinMarketCal events, Reddit, and Telegram, followed by noise filtering, token extraction, temporal weighting, PCA-based feature compression, K-means clustering, and LLM-driven sentiment reasoning. The design is informed by recent cryptocurrency sentiment and forecasting studies that emphasize multimodal signals, structured sentiment construction, and reproducible agent protocols [1, 2, 3, 4, 5]. Experimental results from the repository test suite show 66.7% accuracy on a small Twitter sentiment test set, 90.0% accuracy on CoinGecko-based sentiment analysis, and 80.0% accuracy on price-label classification. These results suggest that LLM4Crypto is a practical and extensible foundation for cryptocurrency sentiment mining and downstream market interpretation.

1 Introduction

Cryptocurrency markets react quickly to news, community narratives, exchange announcements, and opinion leaders' posts. Compared with traditional financial markets, the signal-to-noise ratio is lower and the dominant sentiment can shift within hours. As a result, sentiment analysis is not only a text-mining task but also a market interpretation task that must combine heterogeneous data sources and time-sensitive features. Recent work shows that cross-platform signals can improve forecasting, that sentiment construction and label design strongly affect predictive performance, and that reproducible agent protocols are important when LLMs are used in financial discovery [1, 2, 5].

LLM4Crypto addresses this problem by building a complete analysis loop: raw data acquisition, cleaning and normalization, feature construction, LLM-based sentiment interpretation, and structured output generation. The repository implements the pipeline in a modular form with dedicated folders for collectors, cleaners, analyzers, models, configs, and utilities, together with test and audit scripts for functional validation.

1.1 Motivation

The project is motivated by three practical limitations of conventional crypto sentiment workflows. First, generic social-media collection tends to be broad and noisy, while crypto analysis often requires targeted collection from high-impact accounts. Second, raw text signals alone are insufficient for market analysis because timing, source credibility, and price movement context all matter. Third, many existing pipelines are fragmented, making it difficult to reproduce a complete analysis workflow or validate results systematically. These limitations are consistent with the direction of recent literature, which increasingly combines sentiment, market signals, and stricter evaluation protocols rather than relying on a single text classifier [1, 3, 5].

1.2 Contributions

This paper highlights four contributions:

1. A KOL-oriented collection strategy for cryptocurrency-related social media data.
2. A modular preprocessing and feature engineering pipeline, including temporal decay weighting, influence weighting, PCA compression, and K-means price-label construction.
3. An LLM-based structured sentiment prompt that outputs sentiment, direction, importance, probabilities, keywords, and reasoning.
4. A working experimental and audit framework that validates the pipeline end-to-end with measurable accuracy results.

2 Related Work

Recent work on cryptocurrency sentiment analysis has moved in three directions. First, multimodal analysis has shown that platform type matters: video-based sentiment from TikTok and text-based sentiment from Twitter can exhibit different temporal behaviors, and combining cross-platform signals may improve forecasting accuracy [1]. Second, studies on sentiment representation and label construction show that the way sentiment is aggregated and the way price movement classes are defined can be more important than the forecasting architecture itself [2]. Third, crypto-specific text classification work has explored two-stage predictive statement classification, GPT-assisted annotation, and class balancing to handle noisy and imbalanced social-media data [3].

In addition, multimodal volatility prediction research has begun to treat cryptocurrency forecasting as a fusion problem rather than a purely textual task [4]. Finally, constrained LLM-agent frameworks emphasize auditable hypothesis search and fixed experimental protocols, which is relevant when LLMs are used in financial analysis pipelines that must remain reproducible [5]. These directions motivate the design choices in LLM4Crypto.

3 Motivation

The main design goal of LLM4Crypto is to make crypto sentiment analysis more focused, more robust, and more interpretable. Instead of treating social posts as isolated text samples, the system attaches market context and source metadata to each item. This is especially important in cryptocurrency markets, where posts from exchanges, analysts, and major opinion leaders may carry very different predictive value.

Another motivation is reproducibility. The repository contains clear module boundaries and a set of test scripts such as `run_full_test.py`, `large_scale_test.py`, and `audit_test_results.py`, which makes the system suitable for iterative development and paper-driven evaluation. This follows the broader trend in recent papers that separate model design from experiment protocol and emphasize auditable evaluation [2, 5].

4 Key Principles

4.1 Targeted multi-source collection

The system collects information from multiple sources, including Twitter/X, Telegram, CoinMarketCal, CoinGecko, and Reddit. A notable design choice is to prioritize KOL-style acquisition rather than indiscriminate crawling. In practice, this helps the system focus on accounts and channels more likely to influence market perception. This also aligns with prior studies that show social-media signal quality depends on platform and source selection [1].

4.2 Noise reduction and normalization

Social-media data is inherently messy. The cleaner module performs advertisement filtering, token extraction, Unicode handling, and field standardization. This step improves downstream model stability by reducing irrelevant content and formatting inconsistencies.

4.3 Context-aware feature engineering

The pipeline adds market context through temporal decay weighting and source influence weighting. Recent messages are assigned higher importance, and users with stronger engagement are weighted more heavily. PCA is then used to compress the fused feature space, and K-means clustering is used to convert future return patterns into discrete labels. This feature-construction strategy is directly aligned with recent evidence that sentiment aggregation and label design strongly shape cryptocurrency prediction performance [2].

4.4 LLM-assisted structured reasoning

Rather than using the LLM only as a classifier, the system asks it to produce structured outputs with sentiment orientation, direction prediction, importance score, keyword extraction, and reasoning. This design supports both interpretability and report generation. It is also consistent with the move toward richer crypto-text modeling in predictive statement classification and multimodal sentiment work [1, 3].

5 Design

5.1 System architecture

The codebase is organized into the following modules:

- **Collectors**: `twitter_scweet_collector.py`, `coingecko_collector.py`, `coinmarketcal_collector.py`, `reddit_praw_collector.py`, `telegram_collector.py`
- **Cleaner**: `data_cleaner.py`
- **Analyzers**: `llm_analyzer.py`, `mock_llm_analyzer.py`
- **Models**: `data_models.py`
- **Configs and utilities**: `config.py`, `helpers.py`

A compact view of the workflow is shown below:

> Raw data acquisition → cleaning and normalization → temporal/influence weighting → PCA and K-means processing → LLM sentiment analysis → structured report output

5.2 Data collection

Twitter/X KOL collection. The project defines a crypto KOL taxonomy that includes elites, major exchanges, news media, analysts, and traders. This enables category-aware collection and metadata augmentation at the tweet level.

CoinGecko collection. The CoinGecko collector obtains market-level and coin-level data such as price, market cap, ranking, and trend information. In the repository's large-scale test, this component successfully collected 250 items with 100% field completeness.

Event and community sources. CoinMarketCal is used to collect scheduled crypto events, while Reddit and Telegram provide supplementary community sentiment signals.

5.3 Data cleaning

The cleaner is designed to preserve meaningful observations while removing noisy content. Its main tasks include ad filtering, token extraction, Unicode handling, and schema normalization. According to the project's audit summary, the cleaning stage preserved all tested CoinGecko and Twitter samples while achieving full token extraction readiness for downstream analysis.

5.4 Feature engineering

Let Δt denote the elapsed time between the current analysis point and a message timestamp. The temporal decay weight is defined as

$$w_t = e^{-\lambda \Delta t}, \qquad \lambda = \frac{\ln 2}{T_{1/2}},$$

where $T_{1/2}$ is the half-life parameter.

A simplified influence score can be written as

$$s_{inf} = followers \cdot (1 + engagement\_rate) \cdot \ln(1 + retweets + likes).$$

The engineered features are then combined and compressed via PCA. For market-label construction, K-means clustering is used on future return intervals to generate discrete classes for downward, neutral, and upward movement. This design echoes recent literature showing that PCA-based aggregation and adaptive label construction improve the robustness of cryptocurrency sentiment-driven prediction [2].

5.5 LLM sentiment analysis

The analyzer uses a structured prompt that asks the model to predict sentiment polarity, direction, importance, probability distribution, keywords, and a concise rationale. This multi-field output is more suitable for market analysis than a single-label sentiment tag.

A representative prompt template is:

> You are a cryptocurrency market analyst. Analyze the text and output: Sentiment, Direction, Importance, Probabilities, Keywords, and Reasoning.

5.6 Implementation-Oriented Design Analysis

Unlike purely conceptual cryptocurrency sentiment analysis studies, the design of LLM4Crypto is tightly coupled with the actual implementation logic in the GitHub repository. The system architecture was iteratively refined according to engineering constraints encountered during development.

Collector Layer Design The `collectors/twitter_scweet_collector.py` module implements category-based KOL crawling. Instead of indiscriminately scraping cryptocurrency tweets, the collector first groups accounts into *elites*, *exchanges*, *news*, *analysts*, and *traders*. During collection, the method `collect_from_kols()` dynamically filters accounts according to influence thresholds and attaches metadata fields such as category labels and influence scores to each tweet instance.

This design directly affects downstream feature engineering because tweet credibility is no longer treated uniformly. Compared with previous sentiment pipelines that only process raw text [?], this implementation introduces an additional structured semantic layer.

Cleaning and Robustness Design The module `cleaners/data_cleaner.py` was introduced after multiple failures occurred during Unicode parsing and malformed API responses. Early experiments showed that cryptocurrency tweets frequently contain emojis, ticker symbols, Unicode escape sequences, and malformed URLs. Without normalization, the LLM analyzer generated unstable outputs.

To solve this issue, the cleaner module applies:

- regex-based token extraction for cryptocurrency symbols,
- emoji removal and Unicode normalization,
- advertisement filtering,
- timestamp standardization.

This implementation significantly reduced invalid JSON parsing errors during the interaction with the GLM-4-Flash API.

Analyzer Layer Design The module `analyzers/llm_analyzer.py` implements a multi-dimensional prompt strategy instead of a single-label sentiment classification. The prompt explicitly requires:

- sentiment polarity,
- market direction prediction,
- importance scoring,
- probability estimation,
- reasoning generation.

The reasoning field was added after observing that single-label outputs were difficult to debug during experimental evaluation. By forcing the LLM to produce chain-of-thought-style explanations, the developer could manually inspect whether prediction errors originated from semantic misunderstanding or noisy input data.

6 Experiments

6.1 Experimental setup

The validated pipeline used CoinGecko market data, Twitter data, cleaning/normalization, LLM analysis, and data-quality auditing. The implementation was evaluated with the real API configuration for the LLM backend (glm-4-flash).

6.2 Evaluation metrics

We use three practical metrics:

- sentiment analysis accuracy,
- price-label classification accuracy,
- data completeness and cleaning consistency.

6.3 Results

Table 1 summarizes the main reported results from the project tests.

Table 1: Main experimental results from the repository test summary

Task	Sample Size	Accuracy	Notes
Twitter analysis	192	66.7%	Small test set, bullish over-prediction
CoinGecko analysis	250	90.0%	Structured market data, stable performance
Price-label classification	20	80.0%	Three-class K-means labeling

6.4 Problems encountered and fixes

During experimentation, several practical issues appeared.

1) API instability and rate limits. The collector layer could fail or return incomplete content when external APIs were temporarily unavailable. The solution was to add defensive parsing, explicit type checks, and a fallback strategy that loads historical or cached data when live requests fail.

2) Unicode and emoji noise. Social-media content contained emoji, special symbols, and mixed encodings, which could disturb downstream parsing. The solution was to normalize Unicode, remove irrelevant symbols, and keep only fields required for analysis.

3) Noisy and overly bullish text interpretation. Some neutral or ambiguous tweets were interpreted too positively by the LLM. The solution was to tighten the prompt, emphasize neutral cases, and keep the model output structured so that sentiment, direction, and reasoning could be checked separately.

4) Small labeled sample size. The Twitter test set was intentionally small, which limited the reliability of accuracy claims. The solution was to treat the result as a functionality check rather than a final benchmark, and to use multi-source tests and audit scripts to validate the end-to-end pipeline.

5) Class imbalance in price labels. Upward movement samples were relatively rare in the small test batch. The solution was to use K-means-based regime labeling and to interpret the label results together with feature distributions rather than relying only on raw class counts. This is consistent with recent work that stresses the importance of sentiment construction and label definition [2].

6.5 Observations

The experimental results suggest three findings. First, the LLM performs well on structured or semi-structured market inputs, especially when the prompt is constrained. Second, social-media sentiment is more difficult because short posts often contain sarcasm, ambiguous phrasing, or mixed signals. Third, the pipeline design is robust enough to preserve end-to-end functionality even when one data source is rate-limited. These observations are in line with recent cryptocurrency sentiment studies that combine multiple sources or multiple stages of classification to improve robustness [1, 3, 4].

7 Limitations and Future Work

Despite the positive results, the current system has several limitations. The most important one is dataset scale: the reported Twitter experiment uses only 12 labeled samples in the demonstration set, which is not sufficient for generalization claims. Another limitation is prompt sensitivity: the LLM can over-interpret neutral market commentary as bullish sentiment. A third limitation is temporal granularity: the current pipeline is more suitable for batch analysis than real-time streaming analysis.

Future work will focus on expanding the dataset, refining prompt design for crypto-specific language, improving clustering thresholds, adding a domain lexicon, and integrating real-time streaming support. Another promising direction is to combine LLM reasoning with conventional financial forecasting models to improve both accuracy and interpretability. The constrained-agent and multimodal directions in recent literature provide a useful roadmap for these extensions [1, 4, 5].

8 AI Assistance and Human Review Statement

This manuscript draft was assisted by ChatGPT for language polishing, section organization, and LaTeX drafting. The author conducted manual review, checked the content against the repository implementation, and edited the final version before submission.

9 Conclusion

This paper presents LLM4Crypto, a modular cryptocurrency market sentiment analysis system that integrates multi-source collection, data cleaning, feature engineering, and LLM-based structured reasoning. The repository already demonstrates a working full pipeline and measurable test results, including 66.7% Twitter sentiment accuracy, 90.0% CoinGecko sentiment accuracy, and 80.0% price-label accuracy.

References

[1] C. Liu et al., "Enhancing Cryptocurrency Sentiment Analysis with Multimodal Features," arXiv:2508.15825, 2025.

[2] M. Mohammadi Nezhad et al., "A unified GRU model for cryptocurrency price prediction and harsh price movement detection using enhanced sentiment analysis," *Scientific Reports*, 2026.

[3] M. Shahiki Tash et al., "Decoding Market Emotions in Cryptocurrency Tweets via Predictive Statement Classification with Machine Learning and Transformers," arXiv:2603.24933, 2026.

[4] H. Wang et al., "Multimodal Cryptocurrency Volatility Prediction Analysis Based on LLM," *ScienceDirect / article in press*, 2026.

[5] Y. Huang et al., "From Hypotheses to Factors: Constrained LLM Agents in Cryptocurrency Markets," arXiv:2604.26747, 2026.